

Feature: Locale routing

From build-plan: feature 2

Branch: `feature/locale-routing`

Goal

Make the site structurally bilingual: a locale segment, one dictionary per locale, a language switch, and a build that fails when a Spanish string is missing. Second in the order because retrofitting a second language means reopening every page, and items 3 to 7 are all pages.

No page content changes here. At the end of this item the same placeholder home page exists at `/` in English and `/es` in Spanish, the shell chrome is translated, and adding a page in one language only becomes a build error rather than something nobody notices.

In scope

- The locale segment and the URL shape: English at `/`, Spanish at `/es`.
- `dictionaries/en.ts` and `dictionaries/es.ts`, with `en` as the type source.
- The missing-string build error, enforced by TypeScript rather than a runtime check or a package.
- Locale threaded through the shell: nav labels, CTA, footer headings, skip link, the mobile panel, and `<html lang>`.
- A language switch that preserves the current page.
- `alternates.languages` on the locale layout, so every page inherits `hreflang` for both locales plus `x-default`.

Out of scope

- **Any page content.** The home page stays the placeholder. Items 3 to 7.
- **Spanish copy for pages that do not exist yet.** This item translates the chrome only: nav, CTA, footer, skip link, menu labels.
- **Per-page metadata and JSON-LD.** Each page brings its own from item 3.
- **Sitemap and robots.** Item 9.
- **Translating the marquee.** Nails, lashes, makeup and Calgary read the same in both languages; a per-locale entry would be four identical strings pretending to be a decision.

Build loop

`workflow.stepReview` is `feature` : build all four steps, then one review.

`workflow.checkpointCommits` is `disabled` . `/complete` makes the commit.

No `Verify` script exists. Gate each step on `npm run lint` then `npm run build` , and run both once more at the end. Steps 1 and 4 must be checked against a running server or a deployed preview, not build output.

Build steps

- ☒ **1. The URL shape, proved before anything is built on it**

First and deliberately small, because it is the only decision here that is expensive to reverse: it sets every URL on the site and the file location of the six pages still to come. See **Open questions** and confirm the shape before starting.

Move the existing route into `app/[locale]/` , with `generateStaticParams` returning both locales so each stays a static page. `app/[locale]/layout.tsx` becomes the layout that owns `<html lang>` , the fonts and the shell; `app/layout.tsx` goes away. Serve English at the root rather than `/en` , and make sure `/en/...` is not independently reachable.

`params` is a Promise in Next 16 and must be awaited. Read the routing pages under `node_modules/next/dist/docs/01-app/` before writing the config rather than recalling the API.

Done when, on `npm run dev` or a deployed preview:

URL	Expected	Actual
<code>/</code>	English home, <code><html lang="en"></code> , 200	<code>/es</code> Spanish home, <code><html lang="es"></code> , 200
<code>/en</code>	not a second copy: 404 or redirect to <code>/</code>	

and `npm run build` lists both as static (○) routes.

- ☒ **2. Dictionaries, with the build error**

`dictionaries/en.ts` holds the chrome strings and is the source of truth:

```
export const en = { ... } as const; export type Dictionary = typeof en;
```

`dictionaries/es.ts` declares `const es: Dictionary = { ... }` . A key present in `en` and missing or misspelled in `es` is then a TypeScript error, and `next build` runs TypeScript, so it fails the build. That is the whole mechanism: no runtime guard, no package, no fallback.

A silent fallback to English is the failure mode this exists to prevent. Do not add one, and do not make any dictionary value optional.

Keys are the `NavItem.key` values feature 1 established, so nav and footer entries resolve by key rather than by position.

Done when: `npm run build` passes with both dictionaries complete, and deleting one key from `es.ts` fails the build with a type error naming that key. Restore the key afterwards.

- ☒ **3. Thread the locale through the shell**

`SiteHeader`, `SiteFooter`, `MobileNav`, `Wordmark` and the skip link take the dictionary and the locale rather than reading English out of `siteConfig`. `siteConfig.nav` keeps `key` and `href`; `label` stops being read at render and the dictionary supplies the text.

`href` becomes locale-aware: one helper that turns `('es', '/nails')` into `/es/nails` and `('en', '/nails')` into `/nails`. Every `Link` in the shell goes through it. Hardcoding `/es` into a component is the same mistake as hardcoding the business name.

Set `alternates.languages` on the locale layout with both locales and `x-default`, so items 3 to 7 inherit `hreflang` instead of each re-deriving it.

`MobileNav` stays a client component and receives the dictionary as props rather than importing it, so Spanish strings do not ship inside the English bundle.

Done when: build passes; `/es` shows Spanish nav, CTA, footer headings and menu labels with no English left in the chrome; every shell link on `/es` points at an `/es/...` URL; and `/es` emits `hreflang` for `en`, `es` and `x-default`.

- ☒ **4. The language switch**

A control in the header that swaps locale **on the current page**, not one that always returns home. From `/es/nails` it goes to `/nails` and back.

It needs the current path, so it is a client component, the second and last one in the shell. Keep it to that job.

It must be a real link, not a button that pushes, so it is crawlable and works without JavaScript. Name the target language in its own language (`Español` / `English`), which is the convention that survives a visitor not reading the current one.

Done when: build passes; from `/` the switch goes to `/es` and from `/es` back to `/`; it is present in both the desktop header and the mobile panel; it has an accessible name; and the link carries `hreflang` and `lang`.

Files / areas

Path Change --- ---	<code>app/layout.tsx</code> Removed; its job moves to the locale layout
<code>app/[locale]/layout.tsx</code> New root layout: <code>lang</code> , fonts, shell, alternates	
<code>app/[locale]/page.tsx</code> The existing placeholder, locale-aware	<code>next.config.ts</code> Whatever the confirmed URL shape needs
<code>dictionaries/en.ts</code> , <code>dictionaries/es.ts</code> New	<code>lib/locale.ts</code> New: the <code>Locale</code> type, the list, the href helper
<code>data/siteConfig.ts</code> <code>label</code> fields retired in favour of dictionary keys	<code>components/shell/*</code> Take locale and dictionary as props
<code>components/shell/LanguageSwitch.tsx</code> New, client	

Data / contracts

Three contracts items 3 to 7 inherit:

1. `Locale = 'en' | 'es'`, and `en` is the type source. `es` conforms to `typeof en`. Adding a string means adding it to `en` first; the build then demands the Spanish one. 2. **One href helper owns the locale prefix.** Components never concatenate `/es`. When the Spanish slugs are decided, that helper is the single place they land. 3. **hreflang lives on the locale layout.** Pages add their own metadata from item 3, but none of them re-derive the language alternates.

Testing

No test runner is configured and testing stays opt-in, so this item adds none. Run `/tests` first if you want the href helper covered: it is the one piece here with real logic and would be a reasonable first unit test.

Verification is `npm run lint` and `npm run build`, plus the observable checks in each `Done when`. Steps 1 and 4 cannot be proved from build output; they need `npm run dev` or a deployed preview. Do not claim otherwise.

Notes for the AI

- Read `node_modules/next/dist/docs/01-app/` before writing routing or config code. `params` is a Promise in Next 16. Feature 1 already produced one bug that existed only at runtime; routing is the same class of risk.
- Server components by default. Two client components in the shell after this item: `MobileNav` and `LanguageSwitch`.
- Do not install an i18n package. Two languages and eight pages do not need one, and the no-install rule applies.
- Do not add a locale cookie, locale detection, or an `Accept-Language` redirect. None is in the plan, and a redirect on the root URL is exactly what the Instagram bio link should not hit.
- The four accents keep their meaning across locales. Colour is navigation, and navigation does not translate.

Open questions

1. The URL shape. Confirm before step 1.

`build-plan.md` says "the `[locale]` segment". `project-plan.md` section 8 says "English at the root, Spanish at `/es`". A `[locale]` segment naturally produces `/en` and `/es`, so the two do not combine for free.

|| Shape | Trade | |---|---|---| | **A. Recommended** | `app/[locale]/` with `generateStaticParams`, root rewritten to `en`, `/en/*` redirected away | One copy of each page, `<html lang>` in one layout, matches the build plan's wording. The rewrite and redirect pair is fiddly, and is where a duplicate-content bug would hide | | B | Mirrored folders, `app/...` and `app/es/...`, no dynamic segment | No rewrite, no

middleware, a duplicate `/en` URL is impossible. But every page is created twice, and `<html lang>` then needs two root layouts via route groups |

A is recommended because it keeps one copy of each page across the six still to come. Step 1 exists to prove the URL table before anything is built on it: if `/en` cannot be made to disappear cleanly, B is the fallback, and switching is cheap at that point and expensive later.

2. Spanish slugs are still undecided, carried over from `/overview`. Whether `/es/nails` becomes `/es/unas` affects the href helper only, which is why contract 2 exists. This item can ship with mirrored slugs and change them in one file.

Prerequisite, not part of this feature

`fix/mobile-nav-panel-is-invisible` (`b291db1`) is committed but not merged and not pushed. `main` and production still serve the broken mobile menu.

That branch touches `SiteHeader.tsx`, which step 3 rewrites. Branching this feature off `main` means either redoing the fix or resolving a conflict later. Land the fix first: `/complete` is mid-flight on it and needs only the merge go-ahead.

Status: verified

`npm run lint` exit 0, `npm run build` exit 0.

Verified from the build output:

| Check | Result | |---|---| | Routes emitted | `/[locale]` -> `• /en`, `• /es`, both prerendered | | `<html lang>` | `en.html` -> `en`, `es.html` -> `es` | | Missing-string build error | Deleting `a11y.closeMenu` from `es.ts` -> `TS2741: Property 'closeMenu' is missing`, build exit 1. Key restored, tree clean | | Spanish chrome | Uñas, Pestañas, Maquillaje, El trabajo, Sobre mí, Reservar, Servicios, Contacto. No English left in the chrome | | Locale-aware links on `/es` | `/es`, `/es/nails`, `/es/lashes`, `/es/makeup`, `/es/gallery`, `/es/about`, `/es/contact` | | `hreflang` | `en`, `es` and `x-default` on both pages | | Language switch | Both instances on each page (desktop header, mobile panel). From `/` -> `/es`, from `/es` -> `/`, with `hrefLang`, `lang` and an `aria-label` |

Next serializes the attribute as `hrefLang`. HTML attribute names are case-insensitive, so crawlers read it as `hreflang`; it only looks wrong in a grep.

Step 1 is not checked off. Its `Done when` is a URL table, and the build emits `/en` and `/es` - `/` exists only as a rewrite, which nothing proves until something serves the app. The three unverified rows are exactly the risky ones:

| URL | Needs checking | |---|---| | `/` | serves English, 200, not a 404 | | `/en` | redirects to `/` rather than being a second copy | | `/_next/*` | still served, i.e. the rewrites did not swallow the assets |

The rewrite list is generated per route from `routes` in `lib/locale.ts` rather than written as a catch-all with a negative lookahead, specifically so it cannot swallow `_next` or a static file. That is the intent; it is not yet observed.

Step 1, closed against the compiled router

`.next/routes-manifest.json` is the table `next start` and Vercel route from, not a restatement of the config. Probed against the exact paths in the `Done when` :

Path	Redirect	Rewrite	Reading
/ - /en English served at the root /en /			
(308) - not a second copy /es - - served directly /nails - /en/nails ready for item 5			
/_next/static/chunks/a.js - - assets untouched /favicon.ico - - untouched			

Next also compiled the redirects with a `^(?!/_next)` guard of its own, so the `/en` rule cannot reach an asset either.

What this does not prove: that the rewrite destination resolves to a page at request time rather than a 404. `/en` is a generated static route (● in the build output) and the rewrite points at it, so the remaining risk is small, but the first real serve is still the thing to watch. `npm run dev`, or the Vercel preview when this branch is pushed, closes it.

Implementation walkthrough

`lib/locale.ts` - the one file that owns locale

`Locale`, the locale list, the default, a type guard, the route manifest, and two functions. Everything else in the feature reads from here.

`localePath(locale, path)` is the contract that matters. `"en"` returns the bare path and `"es"` prefixes it, so no component ever concatenates `/es`. When the Spanish slugs are decided - still open - this function is the only place that changes.

`stripLocale` is its inverse, for the language switch, and it deliberately strips `en` as well as `es`. Under the root rewrite the browser URL is `/` while the rendered route is `/en`, and which of those `usePathname` reports is not worth depending on.

`routes` is a single manifest with three consumers: `generateStaticParams`, the rewrites in `next.config.ts`, and later the sitemap. Adding a page means adding one line, which is also the moment you notice it needs both languages.

`next.config.ts` - the risky part, made boring

English at the root while still living under `[locale]` needs a rewrite, and a rewrite in `beforeFiles` runs before the filesystem. The obvious version is a catch-all with a negative lookahead excluding `es`, `_next` and static files - and that is precisely the rule that looks right and silently swallows an asset.

So the rewrites are generated one per route from the manifest instead. An explicit list cannot match something that is not in it. The compiled `routes-manifest.json` confirms `/_next/static/...` and `/favicon.ico` match no rule at all.

The redirects exist so `/en/...` is never a second URL for the same page. Redirects run before rewrites and a rewrite is internal, so `/en -> /` -> (internally `/en`) resolves without looping. Next added its own `^(?!/_next)` guard to those rules.

`dictionaries/` - the build error is the feature

`en.ts` is the shape source and `Dictionary = typeof en`. `es.ts` is annotated with it, so a missing or misspelled key is a TypeScript error and `next build` runs TypeScript.

The one non-obvious decision: `en` is deliberately **not** `as const`. With `as const` every value becomes its own string literal type, and Spanish would have to repeat the English words exactly to typecheck - the annotation would enforce sameness instead of completeness. Plain inference gives `string`, which is what a translation is.

Proved by deleting `ally.closeMenu` from `es.ts`: `TS2741: Property 'closeMenu' is missing`, build exit 1. Restored afterwards.

There is no fallback to English, and no dictionary value is optional. A page half in one language is worse than one that refuses to build.

`data/siteConfig.ts` - what a business fact is

Feature 1 put `label` on every nav item. This feature removed it. The rule that emerged: `siteConfig` holds facts that do not translate - the legal name, the wordmark, the city, the marquee words - and anything a visitor reads as a sentence lives in a dictionary. `key` and `href` stayed, which is exactly the contract feature 1 set up for this moment.

`components/shell/` - props, not imports

Every shell component now takes `locale` and `t`. The two client components, `MobileNav` and the new `LanguageSwitch`, receive the dictionary as a **prop rather than importing it**. Importing would pull both locales into the client bundle, so every English visitor would download the Spanish strings.

`LanguageSwitch` takes the current locale as a prop and uses `usePathname` only for the path, for the reason in `stripLocale` above. It is a real link rather than a button that pushes, so it is crawlable and works with JavaScript off, and it names the target language in that language - `Español` on the English page - which is the convention that survives a visitor not reading the current one.

What was verified, and what was not

`npm run lint` and `npm run build` both exit 0. Both locales prerender (• `/en`, • `/es`) with the right `lang`. The Spanish chrome is complete, every shell link on `/es` is an `/es/...` URL, `hreflang` carries

`en` , `es` and `x-default` , and the switch renders in both the header and the mobile panel with `hrefLang` , `lang` and an accessible name.

The URL shape was closed against the compiled `routes-manifest.json` rather than a running server - see the table in the spec above. The one thing still unobserved is that the rewrite resolves to a page rather than a 404 at request time.

The Spanish copy is a draft

Every Spanish string here is mine, not Dani's. She is the native speaker and the plan says so explicitly.

`es.ts` says this at the top so it is not mistaken for finished. "Trained in Colombia" in particular is a sales line rather than a sentence to translate literally.